

LISA MCP Server

AI-Powered Linux Quality at Microsoft Scale

Connecting any AI assistant to Microsoft's LISA testing framework via the Model Context Protocol

Executive Briefing

The Problem

Linux testing at scale is slow, manual, and tribal

- Microsoft validates **hundreds of Linux distributions** across Azure, Hyper-V, and bare metal
- Engineers spend **hours** searching test cases, writing YAML, and triaging failures by hand
- Critical knowledge lives in senior engineers' heads — onboarding takes **weeks**
- Failure analysis is artisanal: grep logs → read stack traces → guess root cause
- CI/CD pipelines fail silently — no one knows *why* until someone investigates

Outcome: *slower releases, missed regressions, engineer burnout, partner friction*

The Solution — LISA MCP Server

One sentence in. Validated results out.

```
Engineer: "Run all Tier-0 network tests on Ubuntu 24.04 for Azure  
and tell me why anything failed."
```

```
AI Agent:  47 of 49 tests passed.  
 2 failed — root cause: MTU misconfiguration  
after kernel 6.8 update. Recommended fix: ...  
 HTML + Markdown report saved.
```

A Model Context Protocol server that exposes **18 LISA tools** to any AI client — GitHub Copilot, VS Code, Claude, or any future MCP-compatible assistant.

Why MCP? The Universal AI Tool Standard

	Before (custom integrations)	With MCP
Compatibility	One AI client only	Any MCP-compatible AI
Maintenance	N × M integrations	One server, M clients
Ecosystem	Isolated bespoke tooling	Composable with all MCP tools
Future-proof	Rewrite for every new client	Open standard, accelerating adoption

*MCP is the **USB-C for AI tools** — plug any AI into any capability.
Built on MCP means: zero rewrite when the next AI assistant arrives.*

What's Inside — 18 AI-Powered Tools

Category	Tools
 Discovery	<code>discover_test_cases</code> · <code>list_test_areas</code> · <code>search_tests</code> · <code>get_test_case_details</code>
 Generation	<code>generate_test_suite_code</code> · <code>build_runbook</code> · <code>build_tier_runbook_file</code>
 Validation	<code>validate_runbook_file</code> · <code>add_test_to_existing_runbook</code> · <code>check_lisa_environment</code> · <code>get_tier_info</code>
 Execution	<code>run_lisa_tests</code> · <code>parse_test_results</code>
 AI Analysis	<code>analyze_test_run_with_llm</code> · <code>analyze_failure_root_cause</code> · <code>generate_analysis_report</code> · <code>run_and_analyze</code> · <code>list_llm_providers</code>

Each tool callable in natural language — no CLI flags, no YAML hand-editing, no log grepping.

Live Demo — Verified Today

1. Server boots, all 18 tools registered

```
$ python -m lisa_mcp.server
Server : lisa-mcp-server
Tools  : 18
```

2. Build & validate an Azure T1 runbook

```
build_tier_runbook(
    tier="T1", platform_type="azure",
    image="ubuntu jammy 22.04-lts")
# → valid: True · platform: azure
```

3. Generate a brand-new test suite

```
generate_test_suite(
    suite_class_name="DemoNvmeThroughput",
    area="storage", category="performance",
    ...
)
# → 61 lines of LISA-compliant Python
```

4. Parse & summarize JUnit results

```
Total: 3 · Passed · Failed · Duration · ...
(Ready for LLM root-cause analysis)
```

Impact Inside Microsoft

Speed

100×
faster test discovery

30 min → 1 sentence
runbook authoring

2 hrs → 30 sec
failure triage

People

- New engineers productive on **Day 1** (not Week 3)
- Senior engineers freed from repetitive triage

Quality

- Every PR can trigger tier-appropriate tests without CI/CD expertise
- AI surfaces regression patterns humans miss
- Consistent coverage across all Linux distros

Cost

- Fewer escaped bugs reaching production
- Reduced engineer hours on manual testing
- Lower onboarding cost for new hires

Impact Outside Microsoft

Open Source & Ecosystem

- LISA is already open source — this makes it **accessible to everyone**
- Linux distro maintainers can validate Azure / Hyper-V compatibility instantly
- Partners (Canonical, Red Hat, SUSE) can self-serve test validation

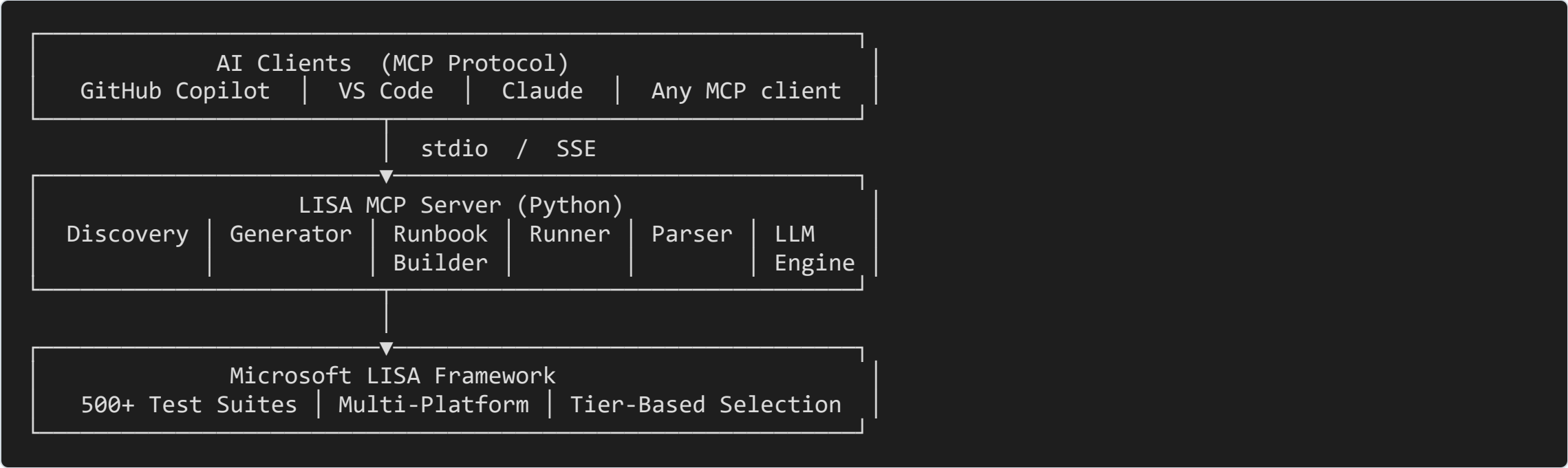
Enterprise Adoption

- Any company running Linux on Azure benefits from the same framework
- ISVs can validate their workloads against Azure infrastructure
- Reduces support tickets by catching issues pre-deployment

Industry Leadership

- Positions Microsoft as the leader in **AI-augmented DevOps**
- Reference architecture for AI-powered testing in any domain
- **Demonstrates MCP as a viable enterprise standard**

Architecture — Simple, Extensible, Local-First



- Local-first
- No infra
- Multi-LLM
- Open source

LLM analysis supports Azure OpenAI · OpenAI · Ollama · LM Studio · Azure AI Foundry.

Before & After

Workflow	Before (manual)	With LISA MCP
Find relevant tests	Grep Python files, read docstrings	"Show network tests for RHEL 9"
Create test config	Hand-write 100+ line YAML	"Build a T1 runbook for Azure"
Write new tests	Copy-paste boilerplate, read docs	"Generate an NVMe throughput test"
Understand failures	Read JUnit + logs manually	"Why did my storage tests fail?"
Onboard new engineer	Two-week shadowing period	"How do I test kernel 6.x on Hyper-V?"
Executive reporting	Manual spreadsheet compilation	Auto-generated HTML / Markdown report

Real-World Scenarios

A — Nightly CI/CD failure triage

"Analyze the failures from last night's T2 run."

*AI: 3 failures — **MTU mismatch after kernel update (critical)**, fio timeout on D4s_v3 (medium), known hugepage flake (low). Recommended fixes attached.*

B — New distro validation

"We need to validate Mariner 3.0 for Azure — what tests should we run?"

*AI: **127 applicable tests** across 8 areas. T0+T1 runbook built. Execute now?*

C — Partner enablement

"Does Ubuntu 24.10 pass Azure network certification?"

*AI: **45 of 47 passed**, 2 warnings (non-blocking). Full report attached.*

Strategic Advantage

Dimension	Without LISA MCP	With LISA MCP
Time to validate a new Linux kernel	2–3 days	2–3 hours
Engineer expertise required	Senior (3+ yrs LISA)	Any level
Cross-team collaboration	Email chains + meetings	Shared AI interface
Partner self-service	Not possible	Fully enabled
AI integration maturity	Scripts & chatbots	Native MCP standard

Bottom line: ship Linux support faster, with higher quality, using fewer senior-engineer hours.

Getting Started — 3 Minutes to Value

```
# 1. Clone
git clone https://github.com/kkkashan/LISA_MCP_Server.git

# 2. Install
cd LISA_MCP_Server && pip install -e .

# 3. Use with any MCP client
# VS Code → Command Palette → MCP: Start Server → lisa
```

- No cloud deployment for the server itself
- No API keys required for discovery / generation / execution
- LLM analysis works with Azure OpenAI today, or any OpenAI-compatible endpoint
- Already shipped: `.vscode/mcp.json` auto-registers in any VS Code workspace

Call to Action

For Engineering Leadership

-  Pilot with a Linux validation team this sprint
-  Integrate into existing CI/CD pipelines
-  Measure reduction in triage time (target: $\geq 80\%$)

For Product & Partner Teams

-  Enable partner self-service testing
-  Use AI-generated reports for stakeholder communication
-  Standardize on MCP for future AI-tool integrations

For the Broader Organization

-  Reference architecture for AI-augmented engineering workflows
-  Extend the pattern to Windows, .NET, Azure SDKs, and beyond
-  Contribute back upstream — grow the MCP and LISA ecosystems

LISA MCP Server

AI × Linux Quality × Developer Productivity

*"The best testing tool is the one engineers actually use.
With AI, they don't even need to know it exists —
they just describe what they want."*

 github.com/kkkashan/LISA_MCP_Server · MIT · Built on MCP